

# Data Pipeline and Model Construction for Graph–Temporal P&L Prediction of Exotic Derivatives

DRAFT — NOT FOR DISTRIBUTION

March 10, 2026

## 1 Introduction

The accurate prediction of daily profit-and-loss (P&L) for portfolios of exotic derivative instruments constitutes a challenging regression problem at the intersection of financial engineering and representation learning. Unlike vanilla options, whose sensitivities are well characterised by low-order Greeks, exotic payoffs exhibit path dependence, discontinuous barrier events, and complex cross-instrument interactions that resist closed-form approximation. Traditional risk systems rely on Monte Carlo re-valuation, which is computationally expensive and provides limited scope for intra-day, portfolio-wide forecasting.

In this work we develop a neural architecture that ingests both the *structural* relationships between trades—encoded as a sparse graph—and the *temporal* dynamics of their mark-to-market valuations—encoded as multivariate time series. A prerequisite for this architecture is a data pipeline that transforms raw market data and trade-level bookings into a set of well-conditioned tensor inputs. The design of this pipeline is non-trivial: the raw data is heterogeneous, high-dimensional, irregularly observed, and heavy-tailed.

This section describes the end-to-end pipeline, which comprises five stages:

- (i) **Elementary P&L construction** (Section 2): extraction and standardisation of per-trade daily P&L series.
- (ii) **Trade graph construction** (Section 3): formation of a sparse  $k$ -nearest-neighbour graph over the trade universe in attribute space.
- (iii) **Attribute encoding** (Section 4): mapping of heterogeneous trade descriptors into fixed-length feature vectors via learned embeddings and normalisation.
- (iv) **Dimensionality reduction** (Section 5): projection of the P&L matrix onto a truncated orthogonal basis that preserves the dominant modes of portfolio covariance.
- (v) **Model input assembly** (Section 6): packaging of the above outputs into the structured tensor collection consumed by the model.

Each stage is designed to be modular: the graph construction, attribute encoding, and dimensionality reduction may be independently reconfigured without altering the model architecture, thereby facilitating rapid experimentation across asset classes and portfolio compositions. A schematic overview of the full pipeline is provided in Figure 1.

## 2 Elementary P&L Construction

### 2.1 Extraction and temporal alignment

Let  $\mathcal{T} = \{1, \dots, T\}$  denote the universe of  $T$  live trades observed over  $S$  business days indexed by  $t \in \{1, \dots, S\}$ . For each trade  $i$ , the front-office risk system provides a daily mark-to-market P&L value  $\tilde{p}_{i,t}$ , computed as the change in the instrument’s net present value between consecutive close-of-business snapshots:

$$\tilde{p}_{i,t} = \text{NPV}_i(t) - \text{NPV}_i(t-1), \quad i \in \mathcal{T}, t \in \{1, \dots, S\}. \quad (1)$$

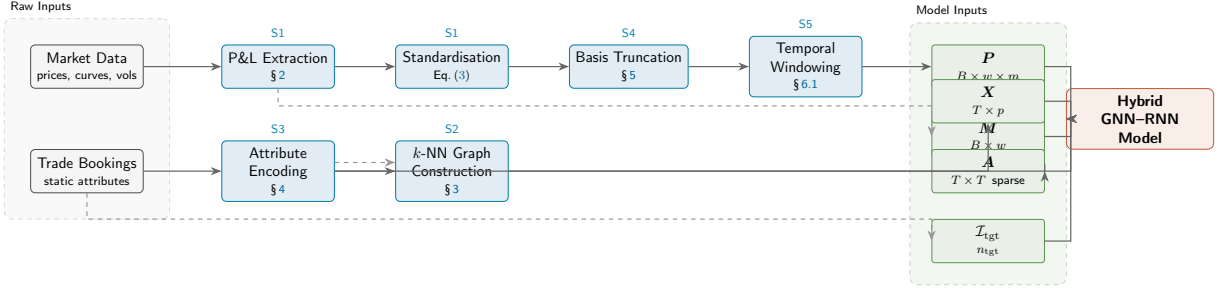


Figure 1: End-to-end data pipeline from raw market data and trade bookings to the structured tensor inputs consumed by the Hybrid GNN–RNN architecture. Solid arrows indicate tensor data flow; dashed arrows indicate configuration dependencies or secondary outputs. Stage labels S1–S5 correspond to Sections 2 to 6, respectively. Tensor shapes are annotated on each output node, where  $T$  is the number of trades,  $B$  the batch size,  $w$  the window length,  $m$  the reduced basis dimension,  $p$  the attribute feature dimension, and  $n_{tgt}$  the number of target trades.

The resulting *elementary P&L matrix*  $\tilde{\mathbf{P}} \in \mathbb{R}^{T \times S}$  is the foundational data object from which all temporal features are derived.

In practice, not every trade is live for the entire observation window. Instruments booked after the start of the window, or maturing before its end, produce incomplete rows. We handle this by zero-padding the missing entries and constructing a binary availability mask  $\mathbf{M} \in \{0, 1\}^{T \times S}$ :

$$M_{i,t} = \begin{cases} 1 & \text{if trade } i \text{ is live on day } t, \\ 0 & \text{otherwise.} \end{cases} \quad (2)$$

The mask is propagated downstream to prevent the model from treating padding as informative signal. This treatment follows the masked sequence modelling paradigm introduced in the natural language processing literature [6] and subsequently adapted to irregularly observed clinical and financial time series [3].

## 2.2 Trade-level standardisation

Raw elementary P&L exhibits two properties that are problematic for gradient-based optimisation. First, the marginal distributions are heavy-tailed, reflecting the non-linear payoff profiles of exotic instruments: barrier knock events, fixing resets, and early exercise can produce daily P&L realisations several standard deviations from the mean. Second, the variance is *heterogeneous* across instrument types—a deep out-of-the-money barrier option may exhibit daily fluctuations orders of magnitude smaller than an at-the-money vanilla on the same underlying [4].

Both properties violate the implicit assumptions of common loss functions and optimisers, leading to poorly conditioned gradients and slow convergence. We address this via a trade-level location-scale standardisation:

$$\bar{p}_{i,t} = \frac{\tilde{p}_{i,t} - \hat{\mu}_i}{\hat{\sigma}_i + \epsilon}, \quad (3)$$

where  $\hat{\mu}_i$  and  $\hat{\sigma}_i$  are robust estimates of the  $i^{\text{th}}$  trade’s central tendency and dispersion, computed over a trailing calibration window of length  $W_{\text{cal}}$ , and  $\epsilon > 0$  is a small constant that prevents degenerate scaling for trades with near-zero P&L variance (*e.g.*, deeply out-of-the-money positions approaching maturity). The use of *robust* estimators—rather than simple sample moments—provides resilience to the outliers that routinely arise in exotic derivative portfolios.

The standardised matrix  $\bar{\mathbf{P}} \in \mathbb{R}^{T \times S}$  has approximately zero mean and unit variance per trade, enabling the recurrent network stream to operate in a numerically well-conditioned regime irrespective of the heterogeneous composition of the portfolio.

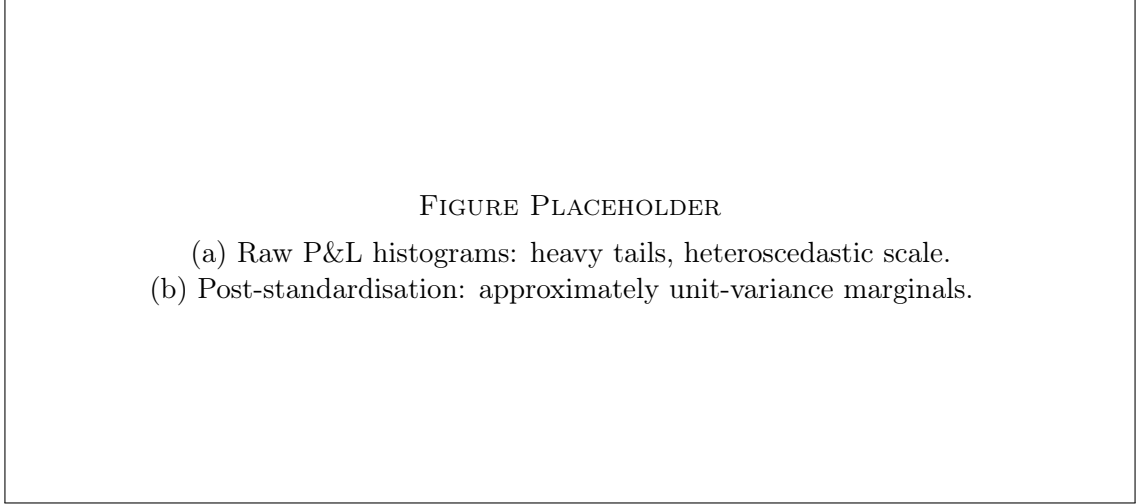


Figure 2: Distributional effect of trade-level standardisation on a synthetic portfolio of 500 trades sampled from a mixture of Student- $t$  distributions with degrees of freedom  $\nu \in \{3, 5, 10\}$  and scale parameters spanning three orders of magnitude.

### 3 Trade Graph Construction

#### 3.1 Motivation and design rationale

The graph neural network stream of the architecture requires an explicit relational structure over the trade universe. Unlike domains in which the graph is given *a priori*—molecular bond graphs, social networks, road networks—the trade graph must be *constructed* from available data. The design of this graph constitutes a critical modelling decision, as it encodes the inductive bias governing which instruments are permitted to exchange information during message passing.

We adopt a  $k$ -nearest-neighbour ( $k$ -NN) construction in the trade attribute space, motivated by the empirical observation that trades sharing structural characteristics—common underlying assets, proximate moneyiness, overlapping tenor—exhibit correlated P&L dynamics. This approach is related to the broader literature on learning graph structure from data [18, 7], though we opt for a fixed, pre-computed graph rather than a jointly learned topology, as the latter introduces a costly bilevel optimisation that is difficult to stabilise at the portfolio scales we consider ( $T \sim 10^3$ – $10^4$ ).

#### 3.2 Construction procedure

Let  $\mathbf{X} \in \mathbb{R}^{T \times p}$  denote the attribute matrix described in Section 4. For each trade  $i$ , we compute a pairwise affinity score  $s(i, j)$  to every other trade  $j \in \mathcal{T}$  and retain the  $k$  trades with the highest affinity as neighbours, forming the neighbour set  $\mathcal{N}_k(i)$ . The affinity function operates over a mixed feature space comprising both categorical and continuous attributes; the precise formulation is domain-specific and is not detailed here.

The resulting directed graph  $\mathcal{G} = (\mathcal{T}, \mathcal{E})$  has  $|\mathcal{E}| = T \cdot k$  edges. We form the weighted adjacency matrix  $\mathbf{A} \in \mathbb{R}^{T \times T}$  by row-normalising the affinity scores:

$$A_{ij} = \frac{s(i, j) \mathbb{1}[j \in \mathcal{N}_k(i)]}{\sum_{j' \in \mathcal{N}_k(i)} s(i, j')}. \quad (4)$$

Row normalisation ensures that the mean aggregation operator in the GraphSAGE message-passing step [14] computes a properly weighted average over neighbours, and that isolated or sparsely connected nodes do not produce unbounded feature magnitudes.

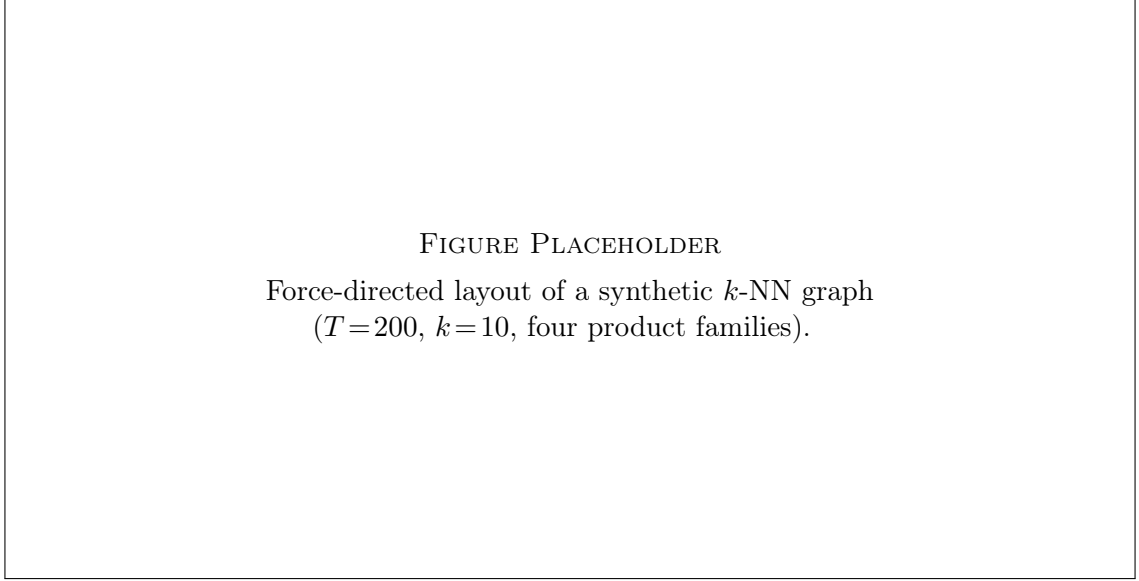


Figure 3: Illustrative  $k$ -NN trade graph generated from four isotropic Gaussian clusters in  $\mathbb{R}^2$ , each representing a distinct product family. Node colour indicates cluster membership. Intra-cluster connectivity is dense, reflecting high attribute-space affinity, while inter-cluster edges arise from trades at cluster boundaries with shared characteristics.

The adjacency matrix is stored in sparse coordinate (COO) format. For a portfolio of  $T = 2,000$  trades with  $k = 30$ , the sparse representation stores  $6 \times 10^4$  non-zero entries in lieu of the  $4 \times 10^6$  elements required by a dense representation—a reduction that is essential for memory-efficient training and inference.

### 3.3 Receptive field and connectivity trade-offs

The neighbour count  $k$  controls the trade-off between locality and connectivity. Small  $k$  yields a sparse graph that preserves fine-grained structural distinctions but may under-connect instruments sharing indirect relationships through intermediary trades. Large  $k$  yields a denser graph that enables broader information propagation at the cost of diluting the neighbourhood signal with less relevant trades.

In a multi-layer GNN of depth  $L$ , the effective receptive field of each node encompasses its  $L$ -hop neighbourhood, the size of which grows as  $\mathcal{O}(k^L)$  in the worst case [21, 1]. Consequently, even moderate values of  $k$  permit the model to aggregate information from structurally distant trades when multiple message-passing layers are employed. We observe empirically that the optimal  $k$  varies with portfolio composition: concentrated books with few product types favour smaller values, whereas diversified multi-asset portfolios benefit from larger  $k$  to capture cross-asset-class dependencies.

## 4 Attribute Encoding

### 4.1 Feature taxonomy

Each trade  $i \in \mathcal{T}$  is described by a vector of static attributes drawn from the trade booking system and the risk engine. These attributes partition naturally into two categories:

- **Categorical features.** Discrete identifiers such as product type (vanilla, barrier, Asian, autocallable, ...), currency pair, exchange venue, and counterparty classification. These possess no natural ordering and may exhibit high cardinality.

- **Continuous features.** Real-valued risk sensitivities (Greeks)—delta, gamma, vega, theta—and contractual quantities such as notional, moneyness, and time-to-maturity. These are informative of the trade’s local sensitivity to market factors but operate on heterogeneous scales and may exhibit non-stationarity (*e.g.*, gamma increases sharply as a barrier is approached).

The precise choice of which attributes to include is a domain engineering decision that depends on the asset class, the available data, and the risk factors of interest [16, 9]. We emphasise that the attribute set should be jointly informative for both the graph construction (Section 3) and the projection layer’s  $k$ -NN transfer mechanism, as both modules rely on attribute-space proximity to generalise across instruments.

## 4.2 Encoding strategy

Categorical features are mapped to dense vectors via learned embedding look-up tables, following established practice in deep learning for tabular data [12, 11]. For a categorical feature with vocabulary size  $V$ , the embedding dimension is set to  $\min(d_{\max}, \lceil V/2 \rceil)$ , where  $d_{\max}$  is a configurable upper bound that balances expressiveness against parameter count.

Continuous features are standardised using training-set statistics and optionally Winsorised at the  $c^{\text{th}}$  percentile to mitigate the influence of extreme realisations in the Greeks that arise during periods of market stress.

The encoded categorical embeddings and standardised continuous features are concatenated to form the per-trade feature vector  $\mathbf{x}_i \in \mathbb{R}^p$ , and the full attribute matrix

$$\mathbf{X} = [\mathbf{x}_1; \mathbf{x}_2; \dots; \mathbf{x}_T] \in \mathbb{R}^{T \times p} \quad (5)$$

serves as input to both the GNN stream (as node features) and the projection layer (for  $k$ -NN-based generalisation to unseen trades).

## 5 Dimensionality Reduction of the P&L Basis

### 5.1 Motivation

The standardised P&L matrix  $\bar{\mathbf{P}} \in \mathbb{R}^{T \times S}$  may possess a large temporal dimension, particularly when extended observation windows are employed to capture slow-moving risk factors. For a portfolio of  $T = 500$  trades observed over  $S = 500$  business days, the recurrent stream would process sequences of length 500 per training window—computationally expensive and susceptible to overfitting, given that the number of free parameters in the recurrent layers scales linearly with the input dimension.

The feasibility of compression rests on a well-documented property of derivative portfolios: the daily P&L of even a large, heterogeneous book is driven by a comparatively small number of market risk factors—equity spot levels, implied volatility surface nodes, yield curve key rates—and the instrument-level P&L vectors lie approximately on a low-dimensional manifold determined by the portfolio’s aggregate factor exposures [5, 19]. Empirically, the leading 10–30 eigenvalues of the P&L covariance matrix typically account for 90–97% of the total variance, with the trailing eigenspace dominated by idiosyncratic noise.

### 5.2 Orthogonal basis truncation

We exploit the low-rank structure by projecting  $\bar{\mathbf{P}}$  onto a truncated orthogonal basis. Let

$$\bar{\mathbf{P}} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top \quad (6)$$

denote the (economy) singular value decomposition of  $\bar{\mathbf{P}}$ , where  $\mathbf{U} \in \mathbb{R}^{T \times r}$ ,  $\mathbf{\Sigma} = \text{diag}(\sigma_1, \dots, \sigma_r)$  with  $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r \geq 0$ ,  $\mathbf{V} \in \mathbb{R}^{S \times r}$ , and  $r = \min(T, S)$ . Define the *retained basis*

$\Phi = [\mathbf{v}_1, \dots, \mathbf{v}_m] \in \mathbb{R}^{S \times m}$  comprising the leading  $m$  right singular vectors. The compressed representation is then

$$\mathbf{P}_{\text{red}} = \bar{\mathbf{P}} \Phi \in \mathbb{R}^{T \times m}. \quad (7)$$

The truncation order  $m$  is selected such that the retained fraction of the total variance exceeds a prescribed threshold  $\tau$ :

$$m = \min \left\{ m' \in \mathbb{N} \mid \frac{\sum_{k=1}^{m'} \sigma_k^2}{\sum_{k=1}^r \sigma_k^2} \geq \tau \right\}. \quad (8)$$

We use the SVD of  $\bar{\mathbf{P}}$  directly, rather than forming the Gram matrix  $\bar{\mathbf{P}}^\top \bar{\mathbf{P}}$  and computing its eigendecomposition, to avoid the squaring of condition numbers that can lead to loss of numerical precision in the trailing singular values [10].

An alternative to the heuristic threshold  $\tau$  is the optimal hard threshold of Gavish and Donoho [8], which provides a distribution-free, asymptotically optimal truncation rule under the spiked covariance model:

$$m^* = \#\{k : \sigma_k > \omega(\beta) \hat{\sigma}_{\text{med}}\}, \quad \omega(\beta) = 0.56 \beta^3 - 0.95 \beta^2 + 1.82 \beta + 1.43, \quad (9)$$

where  $\beta = T/S$  is the aspect ratio and  $\hat{\sigma}_{\text{med}}$  is the median singular value. In our experiments both criteria yield similar truncation orders; we report results under the variance-threshold criterion (8) as it is more widely familiar to practitioners.

### 5.3 Basis selection diagnostics

The choice of  $\tau$  (equivalently,  $m$ ) governs the trade-off between information retention and regularisation. We recommend inspecting the spectral structure of  $\bar{\mathbf{P}}$  prior to fixing  $\tau$ :

- *Cumulative variance plot*:  $\sum_{k=1}^{m'} \sigma_k^2 / \sum_{k=1}^r \sigma_k^2$  versus  $m'$ , with the threshold  $\tau$  indicated by a horizontal reference line.
- *Scree plot*: individual singular values  $\sigma_k$  (log scale) versus  $k$ , revealing the “elbow” that separates systematic from idiosyncratic modes.
- *Reconstruction error*:  $\|\bar{\mathbf{P}} - \bar{\mathbf{P}} \Phi \Phi^\top\|_F / \|\bar{\mathbf{P}}\|_F$  versus  $m$ , illustrating the diminishing returns of additional components.

## 6 Model Input Assembly

### 6.1 Temporal windowing

The compressed P&L matrix  $\mathbf{P}_{\text{red}} \in \mathbb{R}^{T \times m}$  represents the entire observation history as a single snapshot. To form the recurrent input, we extract overlapping windows of length  $w$  along the temporal axis. Each window constitutes one training sample: the model observes the compressed P&L dynamics over  $w$  consecutive business days and produces a prediction for the subsequent period.

Let  $B$  denote the total number of such windows. The recurrent input tensor is

$$\mathbf{P} \in \mathbb{R}^{B \times w \times m}, \quad (10)$$

where  $P_{b,:}$  contains the  $m$ -dimensional compressed P&L for the  $b^{\text{th}}$  window. The window length  $w$  and the stride between consecutive windows are hyperparameters that control the trade-off between temporal context and the number of training samples.

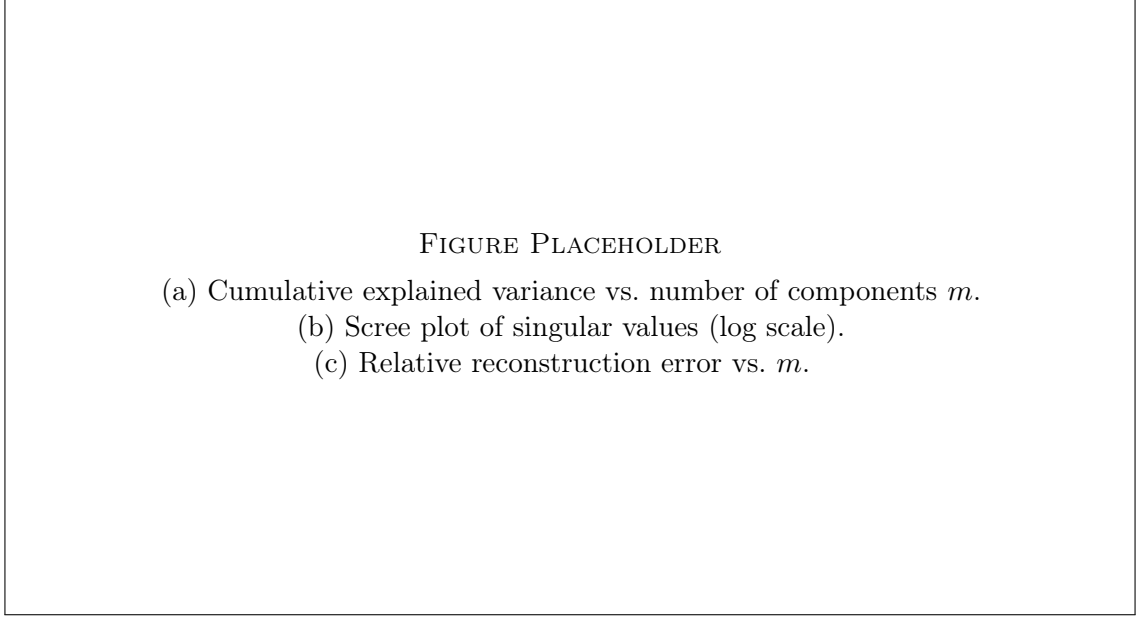


Figure 4: Basis selection diagnostics for a synthetic covariance structure with singular values following a power-law decay  $\sigma_k \propto k^{-\alpha/2}$ ,  $\alpha = 1.5$ , calibrated to reproduce the spectral decay rate observed in derivative portfolio P&L matrices. Dashed horizontal line in (a) indicates  $\tau = 0.95$ , corresponding to  $m = 23$  out of  $S = 500$  components.

Table 1: Model input tensors. The adjacency matrix and attribute features are batch-invariant (shared across all  $B$  training windows).

| Symbol                     | Shape                 | Description                      | Source               |
|----------------------------|-----------------------|----------------------------------|----------------------|
| $\mathbf{X}$               | $T \times p$          | Encoded trade attributes         | Section 4            |
| $\mathbf{P}$               | $B \times w \times m$ | Windowed, basis-reduced P&L      | Section 5            |
| $\mathbf{A}$               | $T \times T$ (sparse) | Row-normalised $k$ -NN adjacency | Section 3            |
| $\mathcal{I}_{\text{tgt}}$ | $n_{\text{tgt}}$      | Target trade indices             | Portfolio definition |
| $\mathbf{M}$               | $B \times w$          | Binary availability mask         | Section 2            |

## 6.2 Structured input collection

The complete model input is a structured collection of five tensors, summarised in Table 1.

The graph  $\mathbf{A}$  and attribute matrix  $\mathbf{X}$  are *batch-invariant*: they describe the static structure of the portfolio and are shared across all  $B$  training windows. This is an important architectural consideration—the GNN stream processes  $\mathbf{X}$  and  $\mathbf{A}$  once to produce node embeddings  $\mathbf{G} \in \mathbb{R}^{T \times d_g}$ , which are subsequently broadcast across the batch dimension when fused with the per-window RNN embeddings.

The target index vector  $\mathcal{I}_{\text{tgt}} \subset \{1, \dots, T\}$  specifies the  $n_{\text{tgt}}$  trades for which the model produces P&L forecasts. In the general case,  $n_{\text{tgt}} \leq T$ : the model leverages the full graph topology during message passing whilst producing outputs only for a designated subset of actively managed positions.

## 6.3 Scalability and deployment considerations

Several practical considerations arise when deploying this pipeline at production scale:

- **Graph recomputation frequency.** The trade graph need not be rebuilt at every training epoch. In practice,  $\mathcal{G}$  is recomputed on a daily or weekly cadence, aligned with the frequency

at which new trades enter the portfolio or existing instruments mature.

- **Inductive handling of new trades.** Trades that appear after the graph was last constructed have no node in  $\mathcal{G}$ . These are accommodated by the projection layer’s inductive transfer mechanism, which computes P&L predictions for unseen instruments by interpolating the outputs of their nearest neighbours in attribute space.
- **Memory footprint.** For large portfolios ( $T > 5,000$ ), a dense  $T \times T$  adjacency matrix would require prohibitive memory. The sparse COO representation reduces storage to  $\mathcal{O}(Tk)$  and enables sparse-only matrix operations throughout the GNN and attention layers, maintaining linear scaling in portfolio size.
- **Reproducibility.** All stochastic components—random seeds for dropout, weight initialisation, and data shuffling—are controlled via a global seed propagated through the pipeline configuration, ensuring bitwise reproducibility of training runs.

## References

- [1] U. Alon and E. Yahav. On the bottleneck of graph neural networks and its practical implications. In *International Conference on Learning Representations (ICLR)*, 2021.
- [2] H. Buehler, L. Gonon, J. Teichmann, and B. Wood. Deep hedging. *Quantitative Finance*, 19(8):1271–1291, 2019.
- [3] Z. Che, S. Purushotham, K. Cho, D. Sontag, and Y. Liu. Recurrent neural networks for multivariate time series with missing values. *Scientific Reports*, 8(1):6085, 2018.
- [4] R. Cont. Empirical properties of asset returns: stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001.
- [5] R. Cont and Y. H. Kan. Statistical modeling of credit default swap portfolios. *SSRN Working Paper*, 2011.
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.
- [7] L. Franceschi, M. Niepert, M. Pontil, and X. He. Learning discrete structures for graph neural networks. In *International Conference on Machine Learning (ICML)*, pages 1972–1982, 2019.
- [8] M. Gavish and D. L. Donoho. The optimal hard threshold for singular values is  $4/\sqrt{3}$ . *IEEE Transactions on Information Theory*, 60(8):5040–5053, 2014.
- [9] P. Glasserman. *Monte Carlo Methods in Financial Engineering*. Springer, 2003.
- [10] G. H. Golub and C. F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.
- [11] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko. Revisiting deep learning models for tabular data. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [12] C. Guo and F. Berkhahn. Entity embeddings of categorical variables. *arXiv preprint arXiv:1604.06737*, 2016.
- [13] N. Halko, P.-G. Martinsson, and J. A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.

- [14] W. Hamilton, Z. Ying, and J. Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 1024–1034, 2017.
- [15] B. Horvath, A. Muguruza, and M. Tomas. Deep hedging under rough volatility. *Quantitative Finance*, 21(8):1–20, 2021.
- [16] J. C. Hull. *Options, Futures, and Other Derivatives*. Pearson, 11th edition, 2022.
- [17] I. T. Jolliffe. *Principal Component Analysis*. Springer, 2nd edition, 2002.
- [18] T. Kipf, E. Fetaya, K.-C. Wang, M. Welling, and R. Zemel. Neural relational inference for interacting systems. In *International Conference on Machine Learning (ICML)*, pages 2688–2697, 2018.
- [19] R. Litterman and J. Scheinkman. Common factors affecting bond returns. *The Journal of Fixed Income*, 1(1):54–61, 1991.
- [20] J. Ruf and W. Wang. Neural networks for option pricing and hedging: a literature review. *Journal of Computational Finance*, 24(1), 2020.
- [21] K. Xu, W. Hu, J. Leskovec, and S. Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019.